

LAPORAN PRAKTIKUM
PRAKTIKUM PENAMBANGAN DATA
PERTEMUAN 1
PENGENALAN NUMPY DAN PANDAS



Disusun oleh:

Nama : Hafidz Rizqullah Prasetya
NIM : 24/535493/SV/24243
Kelas : PL5A1
Dosen Pengampu : Dr. Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Dasar Teori	4
2.1 NumPy	4
2.1.1 Pengertian NumPy dan Array ndarray	4
2.1.2 Dimensi, shape, size, dan dtype	5
2.1.3 Indexing dan slicing	5
2.1.4 Operasi dan manipulasi array	6
2.2 Pandas	6
2.2.1 Pengertian Series dan DataFrame	6
2.2.2 Seleksi, penambahan, dan penghapusan data	7
2.2.3 Grouping dan aggregation	7
2.2.4 Merge, join, dan concatenate	8
2.2.5 Reshaping dan indeks	8
2.2.6 Membaca dataset CSV	8
2.2.7 Keterkaitan NumPy dan pandas	8
Hasil dan Pembahasan	10
3.1 Percobaan Latihan Praktikum	10
3.1.1 Persiapan dan Instalasi Pustaka	10
3.1.2 Impor Library dan Pemeriksaan Versi	10
3.1.3 Pembuatan Array NumPy	11
3.1.4 Indexing dan Slicing Array	12
3.1.5 Konversi Tipe Data dan Reshape	13
3.1.6 Iterasi dan Operasi Dasar Array	14
3.1.7 Join, Split, Search, Sorting, dan Random	15
3.1.8 Operasi Numerik NumPy	16
3.1.9 Membuat dan Menambah DataFrame	17
3.1.10 Menghapus Kolom dan Sorting DataFrame	19
3.1.11 Filtering DataFrame	21
3.1.12 Grouping dan Aggregation	22

3.1.13	Aggregation	23
3.1.14	Merge, Join, Concatenate, dan Indeks	23
3.1.15	Lambda Function dan Membaca Dataset	26
3.2	Tugas dan Analisis Praktikum	28
3.2.1	Tugas 1: Array Acak dan Reshape	28
3.2.2	Tugas 2: Groupby dan Aggregation Gaji	29
3.2.3	Tugas 3: Merge DataFrame Karyawan dan Gaji	30
Kesimpulan		32
Daftar Pustaka		33

Tujuan Praktikum

1. Mahasiswa mampu menjelaskan konsep dasar NumPy, membuat array berdimensi-N, serta melakukan indexing, slicing, reshape, penggabungan, pencarian, pengurutan, dan operasi numerik pada array.
2. Mahasiswa mampu menjelaskan struktur data Series dan DataFrame serta melakukan manipulasi dan transformasi data menggunakan pandas.
3. Mahasiswa mampu menerapkan NumPy dan pandas untuk mengolah, meringkas, dan menganalisis data tabular.

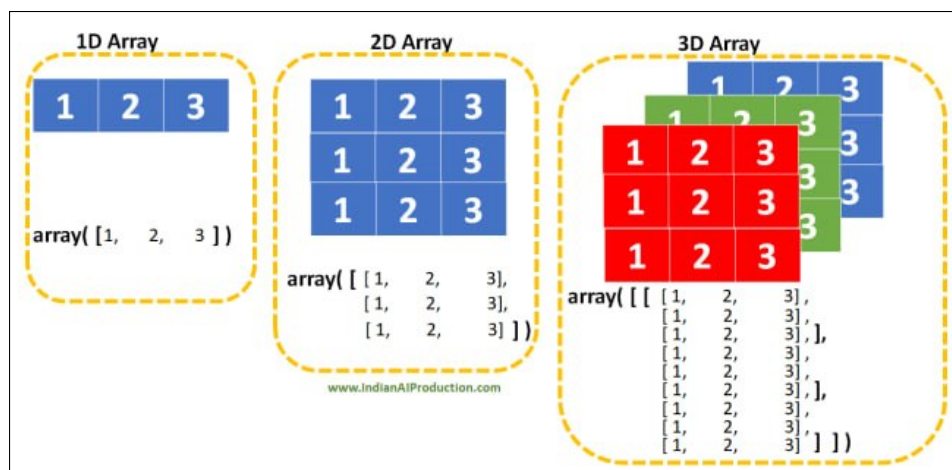
Dasar Teori

2.1 NumPy

2.1.1 Pengertian NumPy dan Array ndarray

NumPy merupakan pustaka Python sumber terbuka yang digunakan untuk komputasi numerik. Struktur data utama NumPy adalah *ndarray*, yaitu array berdimensi-N yang menyimpan elemen dengan tipe data yang homogen. Berbeda dengan list Python yang dapat menyimpan elemen dengan tipe berbeda, array NumPy dirancang untuk menyimpan data sejenis secara efisien. Karakteristik tersebut mendukung operasi numerik pada data dalam jumlah besar [1, 7].

Istilah *ndarray* merupakan singkatan dari *N-dimensional array*. Istilah ini bukan pustaka yang terpisah, melainkan tipe objek array yang digunakan NumPy untuk merepresentasikan data satu dimensi, dua dimensi, atau lebih. Oleh karena itu, pembahasan array NumPy pada praktikum ini merujuk pada objek *ndarray*.



Gambar 1: Ilustrasi array NumPy 1D, 2D, dan 3D.

Array NumPy dapat dibuat dari sekuens Python, seperti list dan *nested list*, menggunakan fungsi `np.array()`. Array satu dimensi dapat digunakan untuk merepresentasikan deret data, sedangkan array dua dimensi dapat digunakan untuk merepresentasikan tabel. Penambahan tingkat kemanunggalan struktur menghasilkan array tiga dimensi atau lebih.

2.1.2 Dimensi, shape, size, dan dtype

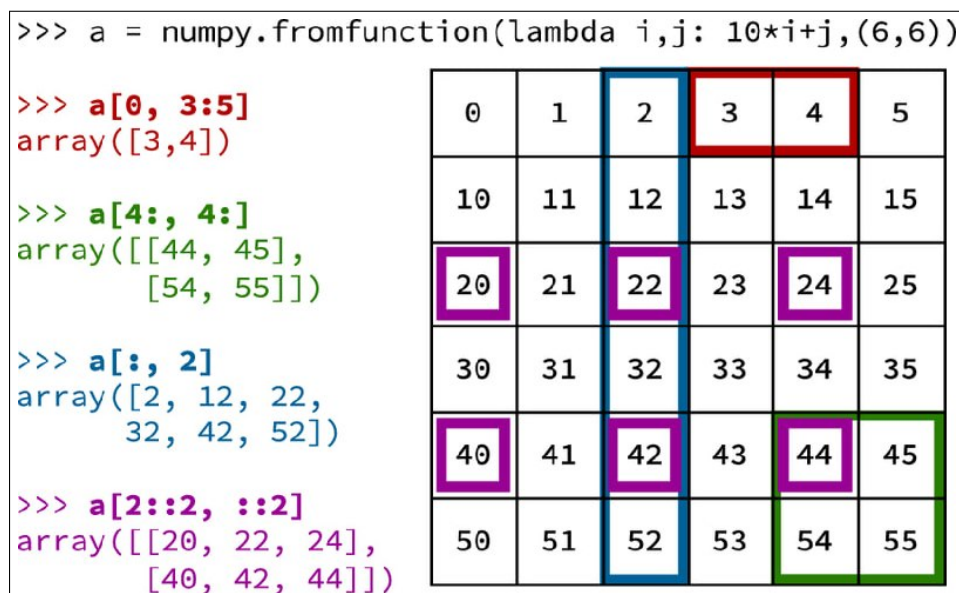
Array mempunyai beberapa atribut penting. Atribut `ndim` menyatakan jumlah dimensi atau sumbu array. Atribut `shape` menghasilkan tuple yang menyatakan jumlah elemen pada setiap sumbu. Atribut `size` menyatakan jumlah seluruh elemen, sedangkan `dtype` menyatakan tipe data elemen array. Sebagai contoh, array dengan shape (2, 3) mempunyai dua baris dan tiga kolom, sehingga jumlah elemennya adalah enam [1].

Pemahaman terhadap shape penting ketika array akan diubah bentuknya atau digabungkan. Operasi `reshape()` hanya dapat dilakukan apabila jumlah elemen sebelum dan sesudah perubahan tetap sama. Dengan demikian, array yang mempunyai dua belas elemen dapat diubah menjadi bentuk (3, 4) atau (2, 6), tetapi tidak menjadi bentuk yang jumlah elemennya bukan dua belas.

2.1.3 Indexing dan slicing

Indexing digunakan untuk mengambil satu elemen berdasarkan posisinya. NumPy menggunakan indeks berbasis nol, sehingga elemen pertama berada pada indeks 0. Pada array dua dimensi, elemen dapat diakses dengan format `array[baris, kolom]`. Pada array dengan lebih banyak dimensi, indeks diberikan untuk setiap sumbu secara berurutan.

Slicing digunakan untuk mengambil sebagian array menggunakan format batas awal, batas akhir, dan langkah, misalnya `array[1:4]`. Batas akhir tidak termasuk dalam hasil slicing. Slicing juga dapat diterapkan pada beberapa sumbu, misalnya `array[2, 1:, 0]`. Pada banyak kasus, hasil slicing NumPy berupa *view* terhadap data asal, bukan salinan baru. Oleh karena itu, perubahan pada hasil slicing dapat memengaruhi array asal [1].



Gambar 2: Ilustrasi indexing dan slicing pada array NumPy 2D.

2.1.4 Operasi dan manipulasi array

NumPy menyediakan operasi element-wise, yaitu operasi yang diterapkan pada elemen-elemen yang bersesuaian, seperti penjumlahan, pengurangan, perkalian, dan pembagian. Fungsi `np.add()` dapat digunakan untuk penjumlahan dua array. Fungsi agregasi seperti `np.sum()`, `np.prod()`, `np.min()`, `np.max()`, dan `np.mean()` digunakan untuk meringkas nilai array. Parameter `axis` dapat digunakan untuk menentukan sumbu tempat operasi dilakukan [1].

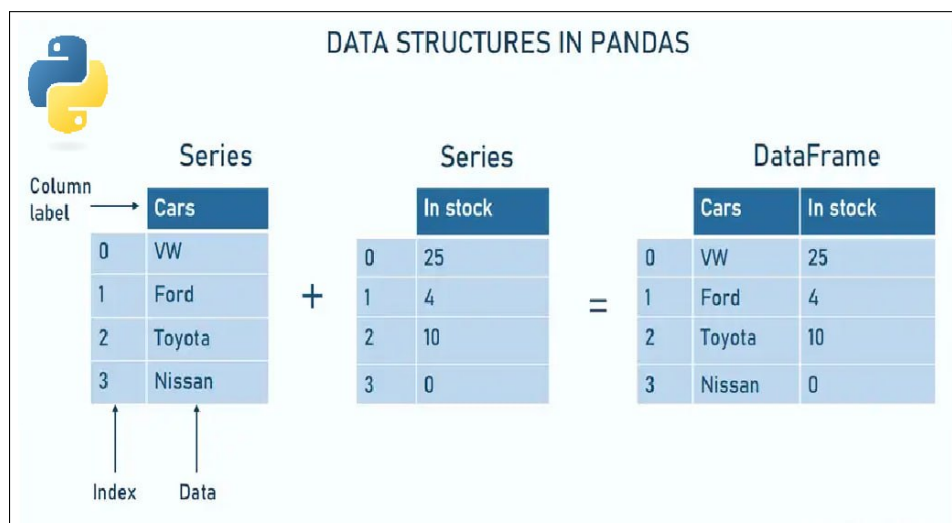
Beberapa fungsi manipulasi yang digunakan dalam praktikum adalah `np.concatenate()` untuk menggabungkan array, `np.array_split()` untuk membaginya menjadi beberapa bagian, `np.where()` untuk mencari posisi elemen yang memenuhi kondisi, dan `np.sort()` untuk mengurutkan elemen. NumPy juga menyediakan modul random untuk menghasilkan bilangan *pseudo-random* dengan bentuk array tertentu. Penggunaan `seed` dapat membuat hasil percobaan dapat diulang.

2.2 Pandas

2.2.1 Pengertian Series dan DataFrame

Pandas merupakan pustaka Python untuk pengolahan dan analisis data tabular. Pandas menyediakan dua struktur data utama, yaitu *Series* dan *DataFrame*. *Series* adalah array satu dimensi yang memiliki label, sedangkan *DataFrame* adalah struktur data dua dimensi yang memiliki baris dan kolom. Setiap kolom *DataFrame* dapat mempunyai tipe data yang berbeda, sehingga *DataFrame* sesuai untuk merepresentasikan dataset tabular [2].

DataFrame dapat dibuat dari list, dictionary, array NumPy, atau data terstruktur lainnya. Label baris disebut indeks dan label kolom digunakan untuk mengenali variabel. Indeks dapat dibuat secara otomatis atau ditentukan secara manual ketika *DataFrame* dibuat.



Gambar 3: Perbandingan struktur Series dan DataFrame pada pandas.

2.2.2 Seleksi, penambahan, dan penghapusan data

Kolom baru dapat ditambahkan menggunakan assignment seperti `df['Status'] = nilai`. Fungsi `insert()` dapat digunakan apabila kolom perlu ditempatkan pada posisi tertentu. Beberapa kolom dapat ditambahkan sekaligus menggunakan `assign()` atau dengan menggabungkan DataFrame kolom baru menggunakan `concat()`.

Kolom dapat dihapus menggunakan `drop()`, `pop()`, atau perintah `del`. Fungsi `drop()` dapat menghapus satu atau beberapa kolom. Perbedaan pentingnya adalah `pop()` menghapus sekaligus mengembalikan kolom yang dihapus, sedangkan `drop()` mengembalikan DataFrame hasil penghapusan apabila tidak digunakan dengan `inplace=True`.

DataFrame dapat diurutkan berdasarkan indeks menggunakan `sort_index()` atau berdasarkan isi kolom menggunakan `sort_values()`. Penyaringan data dilakukan dengan mask Boolean, misalnya `df[df['Gaji'] > 7000000]`. Beberapa kondisi dapat digabungkan dengan operator `&` dan `|`; setiap kondisi perlu diberi tanda kurung. Pandas juga menyediakan `isin()`, `query()`, dan `str.contains()` untuk kebutuhan penyaringan yang lebih spesifik [2].

2.2.3 Grouping dan aggregation

Operasi *groupby* mengikuti pola *split-apply-combine*. Data dibagi menjadi kelompok berdasarkan satu atau beberapa kolom, fungsi tertentu diterapkan pada setiap kelompok, kemudian hasilnya digabungkan kembali. Contohnya, data karyawan dapat dikelompokkan berdasarkan kolom Pekerjaan.

Setelah pengelompokan, fungsi agregasi seperti `mean()`, `sum()`, `min()`, `max()`, dan `std()` dapat digunakan untuk meringkas data numerik. Fungsi `agg()` memungkinkan beberapa fungsi agregasi diterapkan sekaligus, termasuk fungsi yang berbeda untuk kolom yang berbeda. Hasil agregasi membantu membandingkan karakteristik setiap kelompok secara ringkas [3].

	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Gambar 4: Ilustrasi indeks baris, kolom, dan data pada DataFrame pandas.

2.2.4 Merge, join, dan concatenate

Pandas menyediakan beberapa cara untuk menggabungkan DataFrame. Fungsi `merge()` melakukan penggabungan berdasarkan kolom kunci atau indeks, seperti operasi join pada basis data relasional. Parameter `how` dapat bernilai `inner`, `left`, `right`, atau `outer`. Pada *left join*, seluruh baris dari DataFrame sebelah kiri dipertahankan, sedangkan data dari DataFrame kanan ditambahkan jika kuncinya sesuai. Parameter `left_on` dan `right_on` digunakan ketika nama kolom kunci pada kedua DataFrame berbeda. Parameter `suffixes` digunakan untuk membedakan nama kolom yang sama [4].

Fungsi `join()` umumnya menggabungkan DataFrame berdasarkan indeks, sedangkan `concat()` menyatukan beberapa objek pandas pada sumbu tertentu. Dengan `axis=0`, data digabungkan secara vertikal sebagai baris baru; dengan `axis=1`, data digabungkan secara horizontal sebagai kolom baru. Jika label tidak cocok, hasil penggabungan dapat berisi nilai `NaN`.

2.2.5 Reshaping dan indeks

Indeks berfungsi sebagai label untuk baris DataFrame. Fungsi `reset_index(drop=True)` mengembalikan indeks menjadi indeks berurutan dan membuang indeks lama. Sebaliknya, `set_index()` menetapkan salah satu kolom sebagai indeks baru. Pemilihan indeks yang tepat memudahkan akses data menggunakan `loc`.

Pivot table mengubah susunan data sehingga nilai unik pada kolom tertentu menjadi indeks atau kolom baru. Fungsi `pivot_table()` dapat melakukan agregasi ketika terdapat lebih dari satu data pada kombinasi indeks dan kolom. Kebalikan dari format lebar adalah format panjang, yang dapat dibuat dengan `melt()`. Pada operasi ini, kolom yang dipertahankan ditentukan oleh `id_vars`, sedangkan kolom yang diubah menjadi pasangan nama dan nilai ditentukan oleh `value_vars` [5].

2.2.6 Membaca dataset CSV

CSV adalah format teks untuk menyimpan data tabular dengan pemisah antarnilai. Pandas menyediakan `pd.read_csv()` untuk membaca file CSV menjadi DataFrame. Setelah selesai diolah, DataFrame dapat disimpan kembali menggunakan `to_csv()`. Dalam Google Colab, file yang berada di Google Drive dapat diakses setelah Drive di-mount ke lingkungan Colab. Path file kemudian diberikan sebagai argumen kepada `pd.read_csv()` [6].

2.2.7 Keterkaitan NumPy dan pandas

NumPy dan pandas saling melengkapi dalam analisis data. NumPy menyediakan array homogen dan operasi numerik yang efisien, sedangkan pandas menyediakan struktur berlabel untuk mengelola data tabular. DataFrame pandas dapat dibangun dari array NumPy dan nilai

di dalamnya dapat dikonversi ke representasi NumPy menggunakan `to_numpy()`. Kombinasi keduanya memungkinkan proses mulai dari pembentukan data, transformasi, peringkasan, hingga analisis dataset.

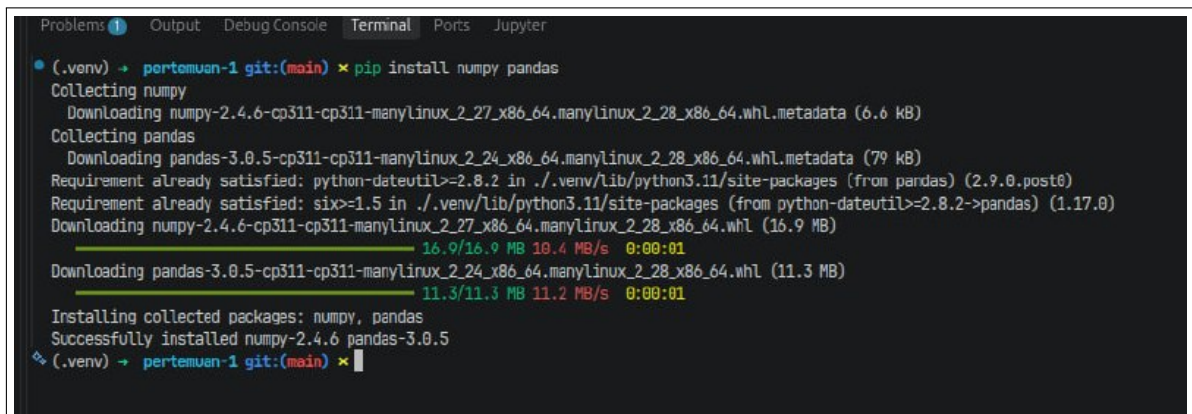
Hasil dan Pembahasan

3.1 Percobaan Latihan Praktikum

Pada bagian ini saya mendokumentasikan langkah-langkah percobaan NumPy dan pandas. Setiap percobaan dilengkapi kode, hasil eksekusi, dan pembahasan singkat.

3.1.1 Persiapan dan Instalasi Pustaka

Pada tahap awal, saya mengaktifkan virtual environment kemudian memasang pustaka NumPy dan pandas menggunakan perintah `pip install numpy pandas`. Proses instalasi berhasil dilakukan pada lingkungan Python 3.11 berbasis Linux. Versi NumPy yang terpasang adalah 2.4.6, sedangkan versi pandas adalah 3.0.5.



```
Problems 1 Output Debug Console Terminal Ports Jupyter
(.venv) → pertemuan-1 git:(main) × pip install numpy pandas
Collecting numpy
  Downloading numpy-2.4.6-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting pandas
  Downloading pandas-3.0.5-cp311-cp311-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in ./venv/lib/python3.11/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.11/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Downloading numpy-2.4.6-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.9 MB)
  16.9/16.9 MB 10.4 MB/s 0:00:01
Downloading pandas-3.0.5-cp311-cp311-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (11.3 MB)
  11.3/11.3 MB 11.2 MB/s 0:00:01
Installing collected packages: numpy, pandas
Successfully installed numpy-2.4.6 pandas-3.0.5
(.venv) → pertemuan-1 git:(main) ×
```

Gambar 1: Instalasi pustaka NumPy dan pandas pada virtual environment.

3.1.2 Impor Library dan Pemeriksaan Versi

Saya mengimpor NumPy dengan alias `np` dan pandas dengan alias `pd`. Pemeriksaan versi dilakukan untuk memastikan kedua pustaka telah terpasang dan dapat digunakan.

```
import numpy as np
import pandas as pd

print('NumPy:', np.__version__)
print('pandas:', pd.__version__)
```

[4] ✓ 0.2s

NumPy: 2.4.6
pandas: 3.0.5

Gambar 2: Impor library dan pemeriksaan versi NumPy serta pandas.

3.1.3 Pembuatan Array NumPy

Saya membuat array NumPy satu dimensi, dua dimensi, dan tiga dimensi menggunakan fungsi `np.array()`. Saya juga membuat array yang berisi elemen bertipe string. Atribut `shape`, `ndim`, dan `dtype` digunakan untuk mengetahui bentuk, jumlah dimensi, dan tipe data array.

A. NumPy

NumPy menggunakan struktur utama bernama `ndarray`. Indeks array dimulai dari 0.

```
# Array 1 dimensi
array_1d = np.array([10, 20, 30, 40, 50])
array_1d

# Shape menunjukkan banyak elemen pada setiap dimensi
print('shape:', array_1d.shape)
print('ndim:', array_1d.ndim)
print('dtype:', array_1d.dtype)

# Array 2 dimensi
array_2d = np.array([[1, 2, 3], [4, 5, 6]])
print(array_2d)
print('shape:', array_2d.shape)

# Array 3 dimensi
array_3d = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])
print(array_3d)
print('shape:', array_3d.shape)

# Array dengan elemen String
array_string = np.array(['Python', 'NumPy', 'Pandas'])
print(array_string)
```

Gambar 3: Pembuatan array NumPy satu, dua, dan tiga dimensi serta array string.

3.1.4 Indexing dan Slicing Array

Saya melakukan indexing pada array satu, dua, dan tiga dimensi. Indexing digunakan untuk mengambil elemen tertentu berdasarkan indeks yang dimulai dari nol. Selain itu, saya menggunakan slicing untuk mengambil sebagian elemen array pada rentang indeks tertentu.

Indexing dan slicing

Gunakan format `array[indeks]` untuk array 1D, `array[baris, kolom]` untuk array 2D, dan tambahkan indeks sesuai jumlah dimensinya.

```
print('array 1D indeks 0:', array_1d[0])
print('array 1D indeks terakhir:', array_1d[-1])
print('array 2D baris 1 kolom 2:', array_2d[1, 2])
print('array 3D [sumbu0, sumbu1, sumbu2]:', array_3d[1, 0, 1])
print('slicing array 1D:', array_1d[1:4])
print('slicing array 2D:', array_2d[:, 1:])
```

```
[0] ✓ 0.0s
"""
array 1D indeks 0: 10
array 1D indeks terakhir: 50
array 2D baris 1 kolom 2: 6
array 3D [sumbu0, sumbu1, sumbu2]: 6
slicing array 1D: [20 30 40]
slicing array 2D: [[2 3]
 [5 6]]
```

```
D> # Contoh array 3D untuk slicing target nilai 3 dan 0
contoh_3d = np.array([
    [[1, 2], [0, 0], [0, 0]],
    [[0, 0], [0, 0], [0, 0]],
    [[8, 9], [3, 4], [0, 5]],
    [[0, 0], [0, 0], [0, 0]]
])
target = contoh_3d[2, 1:, 0]
print('shape:', contoh_3d.shape)
print('hasil slicing [2, 1:, 0]:', target)
```

```
[10] ✓ 0.0s
"""
shape: (4, 3, 2)
hasil slicing [2, 1:, 0]: [3 0]
```

Gambar 4: Indexing dan slicing pada array NumPy satu, dua, dan tiga dimensi.

3.1.5 Konversi Tipe Data dan Reshape

Saya mengubah tipe data array menggunakan `astype()`. Saya juga mengubah bentuk array menggunakan `reshape()`, dengan tetap mempertahankan jumlah seluruh elemen. Untuk mengembalikan array menjadi satu dimensi, saya menggunakan `reshape(-1)` dan `flatten()`.

Konversi tipe data dan reshape

Jumlah seluruh elemen harus tetap sama ketika melakukan `reshape`. Nilai `-1` dapat digunakan agar NumPy menghitung ukuran dimensi secara otomatis.

```
angka = np.array([1, 2, 3, 4])
print('sebelum:', angka.dtype)
print('sesudah:', angka.astype(float).dtype)

array_12 = np.arange(1, 13)
array_3x4 = array_12.reshape(3, 4)
print('reshape 3x4:\n', array_3x4)
print('kembali 1D:', array_3x4.reshape(-1))
print('Flatten:', array_3x4.flatten())
```

[11] ✓ 0.0s

```
sebelum: int64
sesudah: float64
reshape 3x4:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
kembali 1D: [ 1  2  3  4  5  6  7  8  9 10 11 12]
flatten: [ 1  2  3  4  5  6  7  8  9 10 11 12]
```

▷

```
# Perbedaan len() dan shape
array_4x2x3 = np.zeros((4, 2, 3))
print('len:', len(array_4x2x3))
print('shape:', array_4x2x3.shape)
```

[12] ✓ 0.0s

```
len: 4
shape: (4, 2, 3)
```

▷

```
# Iterasi ndarray
for nilai in array_1d:
    print(nilai)

print('Iterasi dengan indeks:')
for i in range(len(array_1d)):
    print(f'indeks {i}: {array_1d[i]}')
```

[13] ✓ 0.0s

```
10
20
30
40
50
Iterasi dengan indeks:
indeks 0: 10
indeks 1: 20
indeks 2: 30
indeks 3: 40
indeks 4: 50
```

Gambar 5: Konversi tipe data, reshape, perbandingan `len()` dan `shape`, serta iterasi array NumPy.

3.1.6 Iterasi dan Operasi Dasar Array

Saya membandingkan fungsi `len()` dengan atribut `shape`. Saya juga melakukan iterasi langsung terhadap array dan iterasi menggunakan indeks. Hasilnya menunjukkan bahwa `len()` hanya mengembalikan ukuran pada sumbu pertama, sedangkan `shape` memberikan ukuran seluruh dimensi.

Konversi tipe data dan reshape

Jumlah seluruh elemen harus tetap sama ketika melakukan `reshape`. Nilai `-1` dapat digunakan agar NumPy menghitung ukuran dimensi secara otomatis.

```
angka = np.array([1, 2, 3, 4])
print('sebelum:', angka.dtype)
print('sesudah:', angka.astype(float).dtype)

array_12 = np.arange(1, 13)
array_3x4 = array_12.reshape(3, 4)
print('reshape 3x4:\n', array_3x4)
print('kembali 1D:', array_3x4.reshape(-1))
print('Flatten:', array_3x4.flatten())
```

[11] ✓ 0.0s

```
sebelum: int64
sesudah: float64
reshape 3x4:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
kembali 1D: [ 1  2  3  4  5  6  7  8  9 10 11 12]
flatten: [ 1  2  3  4  5  6  7  8  9 10 11 12]
```

▶ # Perbedaan len() dan shape

```
array_4x2x3 = np.zeros((4, 2, 3))
print('len:', len(array_4x2x3))
print('shape:', array_4x2x3.shape)
```

[12] ✓ 0.0s

```
len: 4
shape: (4, 2, 3)
```

Iterasi ndarray

```
for nilai in array_1d:
    print(nilai)

print('Iterasi dengan indeks:')
for i in range(len(array_1d)):
    print(f'indeks {i}: {array_1d[i]}')
```

[13] ✓ 0.0s

```
10
20
30
40
50
Iterasi dengan indeks:
indeks 0: 10
indeks 1: 20
indeks 2: 30
indeks 3: 40
indeks 4: 50
```

Gambar 6: Perbandingan len(), shape, dan iterasi array NumPy.

3.1.7 Join, Split, Search, Sorting, dan Random

Saya menggabungkan array menggunakan `np.concatenate()`, membagi array menggunakan `np.array_split()`, mencari elemen menggunakan `np.where()`, serta mengurutkan data menggunakan `np.sort()`. Saya juga menggunakan `np.random` untuk menghasilkan bilangan acak.


```
Join, split, search, sorting, dan operasi numerik

kiri_id = np.array([1, 2, 3])
kanan_id = np.array([4, 5, 6])
print('join 1D:', np.concatenate((kiri_id, kanan_id)))

atas = np.array([[1, 2], [3, 4]])
bawah = np.array([[5, 6], [7, 8]])
print('join axis=0:\n', np.concatenate((atas, bawah), axis=0))
print('join axis=1:\n', np.concatenate((atas, bawah), axis=1))

print('split:', np.array_split(np.arange(1, 10), 3))

join 1D: [1 2 3 4 5 6]
join axis=0:
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
join axis=1:
[[1 2 5 6]
 [3 4 7 8]]
split: [array([1, 2, 3]), array([4, 5, 6]), array([7, 8, 9])]

data = np.array([4, 1, 7, 2, 7, 9])
print('indeks nilai > 5:', np.where(data > 5))
print('indeks nilai 7:', np.where(data == 7))
print('sorting ascending:', np.sort(data))

np.random.seed(42)
random_id = np.random.rand(2)
random_2d = np.random.rand(2, 3)
print('random 1D:', random_id)
print('random 2D:\n', random_2d)

indeks nilai > 5: (array([2, 4, 5]),)
indeks nilai 7: (array([2, 4]),)
sorting ascending: [1 2 4 7 7 9]
random 1D: [0.37454012 0.95871431]
random 2D:
[[0.73199394 0.59865848 0.15601864]
 [0.15599452 0.85808361 0.86617615]]

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print('add:', np.add(a, b))
print('sum seluruh elemen:', np.sum(a))
print('product seluruh elemen:', np.prod(a))

add: [5 7 9]
sum seluruh elemen: 6
product seluruh elemen: 6
```

Gambar 7: Operasi join, split, search, sorting, random, dan operasi numerik pada NumPy.

3.1.8 Operasi Numerik NumPy

Saya melakukan penjumlahan elemen dengan `np.add()`, menghitung total elemen menggunakan `np.sum()`, dan menghitung hasil perkalian seluruh elemen dengan `np.prod()`.

```
Join, split, search, sorting, dan operasi numerik

kiri_id = np.array([1, 2, 3])
kanan_id = np.array([4, 5, 6])
print('join 1D:', np.concatenate((kiri_id, kanan_id)))

atas = np.array([[1, 2], [3, 4]])
bawah = np.array([[5, 6], [7, 8]])
print('join axis=0:\n', np.concatenate((atas, bawah), axis=0))
print('join axis=1:\n', np.concatenate((atas, bawah), axis=1))

print('split:', np.array_split(np.arange(1, 10), 3))

join 1D: [1 2 3 4 5 6]
join axis=0:
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
join axis=1:
[[1 2 5 6]
 [3 4 7 8]]
split: [array([1, 2, 3]), array([4, 5, 6]), array([7, 8, 9])]

data = np.array([4, 1, 7, 2, 7, 9])
print('indeks nilai > 5:', np.where(data > 5))
print('indeks nilai 7:', np.where(data == 7))
print('sorting ascending:', np.sort(data))

np.random.seed(42)
random_1d = np.random.rand(2)
random_2d = np.random.rand(2, 3)
print('random 1D:', random_1d)
print('random 2D:\n', random_2d)

indeks nilai > 5: (array([2, 4, 5]),)
indeks nilai 7: (array([2, 4]),)
sorting ascending: [1 2 4 7 7 9]
random 1D: [0.37454012 0.95871431]
random 2D:
[[0.73199394 0.59865848 0.15601864]
 [0.15599452 0.85808361 0.86617615]]

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print('add:', np.add(a, b))
print('sum seluruh elemen:', np.sum(a))
print('product seluruh elemen:', np.prod(a))

add: [5 7 9]
sum seluruh elemen: 6
product seluruh elemen: 6
```

Gambar 8: Operasi numerik pada array NumPy.

3.1.9 Membuat dan Menambah DataFrame

Saya membuat DataFrame dari list dengan kolom dan indeks tertentu. Selanjutnya, saya menambahkan kolom menggunakan assignment, `insert()`, dan `assign()`. DataFrame digunakan untuk merepresentasikan data karyawan dalam bentuk tabel.

B. DataFrame Pandas

`DataFrame` adalah struktur data tabel dua dimensi pada pandas.

```
# Membuat DataFrame dari list
df = pd.DataFrame([
    ['Andi', 'Programmer', 8000000],
    ['Budi', 'Analis', 7000000],
    ['Citra', 'Programmer', 9000000]
], columns=['Nama', 'Pekerjaan', 'Gaji'], index=['karyawan-1', 'karyawan-2', 'karyawan-3'])
df
```

[16] ✓ 0.0s

	Nama	Pekerjaan	Gaji
karyawan-1	Andi	Programmer	8000000
karyawan-2	Budi	Analis	7000000
karyawan-3	Citra	Programmer	9000000

```
# Menambah kolom dengan assignment, insert, assign, dan dictionary
df_tambah = df.copy()
df_tambah['Status'] = ['Tetap', 'Kontrak', 'Tetap']
df_tambah.insert(1, 'Kota', ['Sleman', 'Bantul', 'Kota Yogyakarta'])
df_tambah = df_tambah.assign(
    Pengalaman=[2, 1, 3], Aktif=[True, True, True]
)
df_tambah
```

[17] ✓ 0.0s

	Nama	Kota	Pekerjaan	Gaji	Status	Pengalaman	Aktif
karyawan-1	Andi	Sleman	Programmer	8000000	Tetap	2	True
karyawan-2	Budi	Bantul	Analis	7000000	Kontrak	1	True
karyawan-3	Citra	Kota Yogyakarta	Programmer	9000000	Tetap	3	True

```
# Menambah beberapa kolom sekaligus dengan dictionary
df_multi = df.copy()
kolom_baru = pd.DataFrame({
    'Status': ['Tetap', 'Kontrak', 'Tetap'],
    'Bonus': [1000000, 500000, 1200000]
}, index=df_multi.index)
df_multi = pd.concat([df_multi, kolom_baru], axis=1)
df_multi
```

[18] ✓ 0.0s

	Nama	Pekerjaan	Gaji	Status	Bonus
karyawan-1	Andi	Programmer	8000000	Tetap	1000000
karyawan-2	Budi	Analis	7000000	Kontrak	500000
karyawan-3	Citra	Programmer	9000000	Tetap	1200000

Gambar 9: Pembuatan DataFrame dan penambahan kolom menggunakan assignment, insert(), assign(), dan concat().

3.1.10 Menghapus Kolom dan Sorting DataFrame

Saya menghapus kolom menggunakan `drop()`, `pop()`, dan perintah `del`. Setelah itu, saya mengurutkan baris berdasarkan indeks dan mengurutkan data berdasarkan nilai kolom gaji menggunakan `sort_values()`.

```

# Menghapus kolom dengan drop, pop, dan del
df_hapus = df_multi.copy()
df_hapus.drop(columns='Bonus', inplace=True)
status_dihapus = df_hapus.pop('Status')
df_hapus['KolomSementara'] = 1
del df_hapus['KolomSementara']
print(df_hapus)
print('Kolom hasil pop:', status_dihapus.tolist())

df_multi_kolom = df_multi.drop(columns=['Status', 'Bonus'])
df_multi_kolom

```

[19] ✓ 0.0s

```

---
      Nama  Pekerjaan  Gaji
karyawan-1  Andi  Programmer  8000000
karyawan-2  Budi    Analis  7000000
karyawan-3  Citra  Programmer  9000000
Kolom hasil pop: ['Tetap', 'Kontrak', 'Tetap']

```

	Nama	Pekerjaan	Gaji
karyawan-1	Andi	Programmer	8000000
karyawan-2	Budi	Analis	7000000
karyawan-3	Citra	Programmer	9000000

```

# Sorting berdasarkan indeks dan nilai
df_sort = df_multi.copy()
df_sort.index = [3, 1, 2]
print('sort index baris:')
display(df_sort.sort_index())
print('sort index kolom:')
display(df_sort.sort_index(axis=1))
print('sort berdasarkan Gaji descending:')
display(df_sort.sort_values(by='Gaji', ascending=False))

```

[20] ✓ 0.0s

```

---
sort index baris:

```

	Nama	Pekerjaan	Gaji	Status	Bonus
1	Budi	Analis	7000000	Kontrak	500000
2	Citra	Programmer	9000000	Tetap	1200000
3	Andi	Programmer	8000000	Tetap	1000000

```

---
sort index kolom:

```

	Bonus	Gaji	Nama	Pekerjaan	Status
3	1000000	8000000	Andi	Programmer	Tetap
1	500000	7000000	Budi	Analis	Kontrak
2	1200000	9000000	Citra	Programmer	Tetap

```

---
sort berdasarkan Gaji descending:

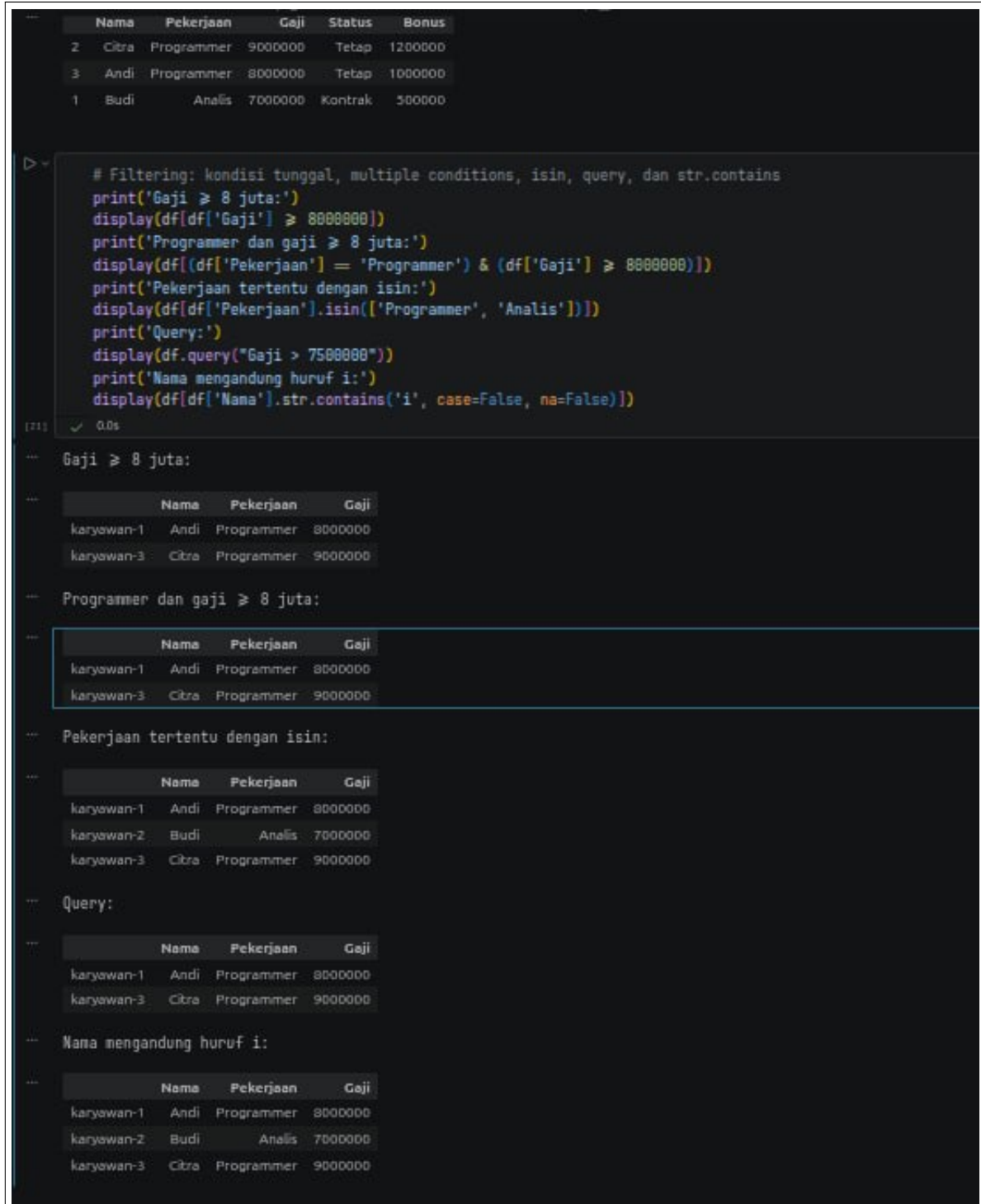
```

	Nama	Pekerjaan	Gaji	Status	Bonus
2	Citra	Programmer	9000000	Tetap	1200000
3	Andi	Programmer	8000000	Tetap	1000000
1	Budi	Analis	7000000	Kontrak	500000

Gambar 10: Penghapusan kolom dengan drop(), pop(), dan del serta pengurutan DataFrame berdasarkan indeks, kolom, dan gaji.

3.1.11 Filtering DataFrame

Saya melakukan filtering menggunakan kondisi Boolean, beberapa kondisi dengan operator logika, `isin()`, `query()`, dan `str.contains()`. Filtering membantu saya menampilkan hanya baris yang memenuhi kriteria tertentu.



```
# Filtering: kondisi tunggal, multiple conditions, isin, query, dan str.contains
print('Gaji ≥ 8 juta:')
display(df[df['Gaji'] ≥ 8000000])
print('Programmer dan gaji ≥ 8 juta:')
display(df[(df['Pekerjaan'] == 'Programmer') & (df['Gaji'] ≥ 8000000)])
print('Pekerjaan tertentu dengan isin:')
display(df[df['Pekerjaan'].isin(['Programmer', 'Analis'])])
print('Query:')
display(df.query("Gaji > 7500000"))
print('Nama mengandung huruf i:')
display(df[df['Nama'].str.contains('i', case=False, na=False)])
```

Gaji ≥ 8 juta:

	Nama	Pekerjaan	Gaji
karyawan-1	Andi	Programmer	8000000
karyawan-3	Citra	Programmer	9000000

Programmer dan gaji ≥ 8 juta:

	Nama	Pekerjaan	Gaji
karyawan-1	Andi	Programmer	8000000
karyawan-3	Citra	Programmer	9000000

Pekerjaan tertentu dengan isin:

	Nama	Pekerjaan	Gaji
karyawan-1	Andi	Programmer	8000000
karyawan-2	Budi	Analisis	7000000
karyawan-3	Citra	Programmer	9000000

Query:

	Nama	Pekerjaan	Gaji
karyawan-1	Andi	Programmer	8000000
karyawan-3	Citra	Programmer	9000000

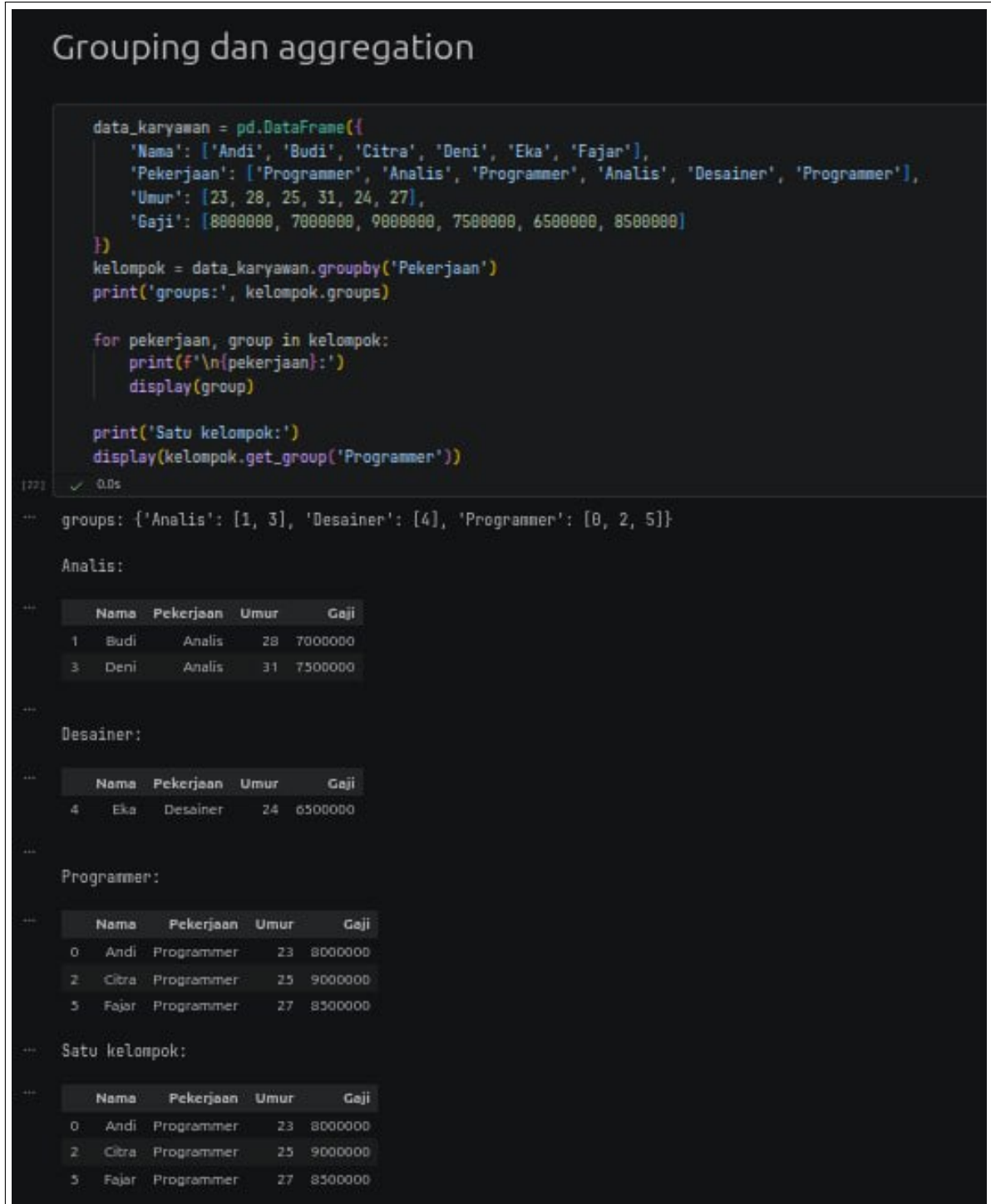
Nama mengandung huruf i:

	Nama	Pekerjaan	Gaji
karyawan-1	Andi	Programmer	8000000
karyawan-2	Budi	Analisis	7000000
karyawan-3	Citra	Programmer	9000000

Gambar 11: Filtering DataFrame menggunakan kondisi Boolean, `isin()`, `query()`, dan `str.contains()`.

3.1.12 Grouping dan Aggregation

Saya mengelompokkan data karyawan berdasarkan kolom Pekerjaan menggunakan `groupby()`. Setelah itu, saya menerapkan fungsi agregasi seperti rata-rata, jumlah, nilai minimum, maksimum, dan standar deviasi untuk meringkas data pada setiap kelompok.



The screenshot shows a Jupyter Notebook titled "Grouping dan aggregation". It contains a Python script that creates a DataFrame of employee data, groups it by job title, and displays the groups. The output shows three groups: Analis, Desainer, and Programmer. The Programmer group is further detailed with a table of its members.

```
data_karyawan = pd.DataFrame({
    'Nama': ['Andi', 'Budi', 'Citra', 'Deni', 'Eka', 'Fajar'],
    'Pekerjaan': ['Programmer', 'Analis', 'Programmer', 'Analis', 'Desainer', 'Programmer'],
    'Umur': [23, 28, 25, 31, 24, 27],
    'Gaji': [8000000, 7000000, 9000000, 7500000, 6500000, 8500000]
})
kelompok = data_karyawan.groupby('Pekerjaan')
print('groups:', kelompok.groups)

for pekerjaan, group in kelompok:
    print(f'\n{pekerjaan}:')
    display(group)

print('Satu kelompok:')
display(kelompok.get_group('Programmer'))
```

groups: {'Analis': [1, 3], 'Desainer': [4], 'Programmer': [0, 2, 5]}

Analis:

	Nama	Pekerjaan	Umur	Gaji
1	Budi	Analis	28	7000000
3	Deni	Analis	31	7500000

Desainer:

	Nama	Pekerjaan	Umur	Gaji
4	Eka	Desainer	24	6500000

Programmer:

	Nama	Pekerjaan	Umur	Gaji
0	Andi	Programmer	23	8000000
2	Citra	Programmer	25	9000000
5	Fajar	Programmer	27	8500000

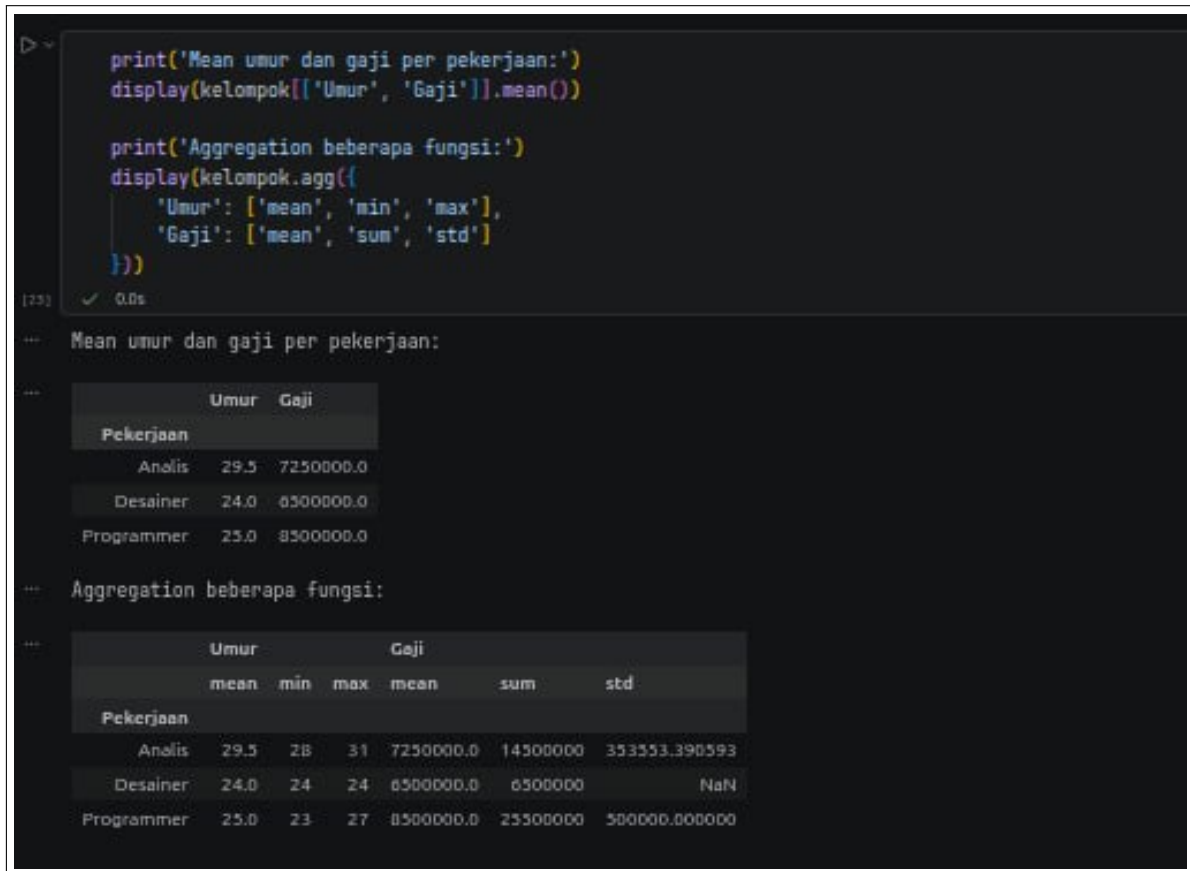
Satu kelompok:

	Nama	Pekerjaan	Umur	Gaji
0	Andi	Programmer	23	8000000
2	Citra	Programmer	25	9000000
5	Fajar	Programmer	27	8500000

Gambar 12: Pengelompokan data karyawan berdasarkan pekerjaan dan pengambilan kelompok Programmer.

3.1.13 Aggregation

Saya menerapkan aggregation pada hasil pengelompokan data untuk menghitung rata-rata, minimum, maksimum, jumlah, dan standar deviasi.



```
print('Mean umur dan gaji per pekerjaan:')
display(kelompok[['Umur', 'Gaji']].mean())

print('Aggregation beberapa fungsi:')
display(kelompok.agg({
    'Umur': ['mean', 'min', 'max'],
    'Gaji': ['mean', 'sum', 'std']
}))
```

Mean umur dan gaji per pekerjaan:

	Umur	Gaji
Pekerjaan		
Analisis	29.5	7250000.0
Desainer	24.0	6500000.0
Programmer	25.0	8500000.0

Aggregation beberapa fungsi:

	Umur			Gaji		
	mean	min	max	mean	sum	std
Pekerjaan						
Analisis	29.5	28	31	7250000.0	14500000	353553.390593
Desainer	24.0	24	24	6500000.0	6500000	NaN
Programmer	25.0	23	27	8500000.0	25500000	500000.000000

Gambar 13: Hasil aggregation rata-rata, minimum, maksimum, jumlah, dan standar deviasi berdasarkan pekerjaan.

3.1.14 Merge, Join, Concatenate, dan Indeks

Saya menggabungkan DataFrame menggunakan `merge()` berdasarkan kolom kunci, menggunakan `join()` berdasarkan indeks, serta menggunakan `concat()` untuk menggabungkan data secara vertikal dan horizontal. Saya juga mengatur ulang indeks menggunakan `reset_index()` dan menetapkan kolom indeks menggunakan `set_index()`.

Merge, join, concatenate, dan indeks

```
karyawan = pd.DataFrame({
    'Emp_id': ['E01', 'E02', 'E03'],
    'Nama': ['Andi', 'Budi', 'Citra'],
    'Divisi': ['Data', 'IT', 'Data']
})

gaji = pd.DataFrame({
    'ID': ['E01', 'E02', 'E04'],
    'Gaji': [8000000.0, 7000000.0, 6000000.0],
    'Status': ['Tetap', 'Kontrak', 'Tetap']
})

hasil_merge = pd.merge(
    karyawan, gaji, left_on='Emp_id', right_on='ID',
    how='left', sort=True, suffixes=('_karyawan', '_gaji')
)
hasil_merge
```

[74] ✓ 0.0s

	Emp_id	Nama	Divisi	ID	Gaji	Status
0	E01	Andi	Data	E01	8000000.0	Tetap
1	E02	Budi	IT	E02	7000000.0	Kontrak
2	E03	Citra	Data	NaN	NaN	NaN

```
# Join berbasis indeks
karyawan_index = karyawan.set_index('Emp_id')
gaji_index = gaji.set_index('ID')
hasil_join = karyawan_index.join(gaji_index, how='left', rsuffix='_gaji')
hasil_join
```

[75] ✓ 0.0s

	Nama	Divisi	Gaji	Status
Emp_id				
E01	Andi	Data	8000000.0	Tetap
E02	Budi	IT	7000000.0	Kontrak
E03	Citra	Data	NaN	NaN

Gambar 14: Penggabungan DataFrame menggunakan merge() dan join() dengan left join.

```
data_tambahan = pd.DataFrame({
    'Nama': ['Deni'], 'Pekerjaan': ['Tester'], 'Gaji': [6500000]
})
print('Concatenate vertikal:')
display(pd.concat([df, data_tambahan], axis=0, ignore_index=True))

kiri = pd.DataFrame({'Nama': ['Andi', 'Budi'], 'Kota': ['Sleman', 'Bantul']})
kanan = pd.DataFrame({'Gaji': [8000000, 7000000], 'Status': ['Tetap', 'Kontrak']})
print('Concatenate horizontal:')
display(pd.concat([kiri, kanan], axis=1))

berbeda_kolom = pd.DataFrame({'Nama': ['Eka'], 'Bonus': [5000000]})
print('Concatenate outer menghasilkan NaN pada kolom yang tidak tersedia:')
display(pd.concat([kiri, berbeda_kolom], join='outer', ignore_index=True))
```

[78] ✓ 0.0s

Concatenate vertikal:

	Nama	Pekerjaan	Gaji
0	Andi	Programmer	8000000
1	Budi	Analisis	7000000
2	Citra	Programmer	9000000
3	Deni	Tester	6500000

Concatenate horizontal:

	Nama	Kota	Gaji	Status
0	Andi	Sleman	8000000	Tetap
1	Budi	Bantul	7000000	Kontrak

Concatenate outer menghasilkan NaN pada kolom yang tidak tersedia:

	Nama	Kota	Bonus
0	Andi	Sleman	NaN
1	Budi	Bantul	NaN
2	Eka	NaN	500000.0

Gambar 15: Penggabungan DataFrame secara vertikal, horizontal, dan outer menggunakan concat().

```

# Reset index dan set index
gabungan = pd.concat([karyawan, karyawan], ignore_index=True)
print('reset saat concat:')
display(gabungan)

reset = hasil_merge.reset_index(drop=True)
print('reset_index(drop=True):')
display(reset)

dengan_index = hasil_merge.set_index('Emp_id')
print('set_index dan akses loc:')
display(dengan_index.loc['E02'])

```

[27] ✓ 0.0s

reset saat concat:

	Emp_id	Nama	Divisi
0	E01	Andi	Data
1	E02	Budi	IT
2	E03	Citra	Data
3	E01	Andi	Data
4	E02	Budi	IT
5	E03	Citra	Data

reset_index(drop=True):

	Emp_id	Nama	Divisi	ID	Gaji	Status
0	E01	Andi	Data	E01	8000000.0	Tetap
1	E02	Budi	IT	E02	7000000.0	Kontrak
2	E03	Citra	Data	NaN	NaN	NaN

set_index dan akses loc:

Nama	Budi
Divisi	IT
ID	E02
Gaji	7000000.0
Status	Kontrak
Name: E02, dtype: object	

Gambar 16: Reset index, set index, dan akses data menggunakan loc pada DataFrame.

3.1.15 Lambda Function dan Membaca Dataset

Saya menggunakan lambda function untuk melakukan operasi singkat dan memberikan kategori berdasarkan nilai. Dataset CSV dibaca menggunakan `pd.read_csv()` dan hasil pengolahan dapat disimpan kembali menggunakan `to_csv()`.

Pivot table, melt, dan lambda

```
data_pivot = pd.DataFrame({
    'Nama': ['Andi', 'Budi', 'Citra', 'Deni', 'Eka', 'Fajar'],
    'Gender': ['L', 'L', 'P', 'L', 'P', 'L'],
    'Pekerjaan': ['Programmer', 'Analis', 'Programmer', 'Analis', 'Dessainer', 'Programmer'],
    'Gaji': [8, 7, 9, 7.5, 6.5, 8.5]
})

pivot = pd.pivot_table(
    data_pivot, values='Gaji', index='Pekerjaan',
    columns='Gender', aggfunc=[np.sum, np.mean]
)

pivot
```

[78] ✓ 0.0s

Gender	sum		mean	
	L	P	L	P
Pekerjaan				
Analis	14.5	NaN	7.25	NaN
Dessainer	NaN	6.5	NaN	6.5
Programmer	16.5	9.0	8.25	9.0

```
nilai = pd.DataFrame({
    'Nama': ['Andi', 'Budi', 'Citra'],
    'Matematika': [88, 75, 90],
    'Bahasa': [85, 80, 88]
})

nilai_long = pd.melt(
    nilai, id_vars=['Nama'],
    value_vars=['Matematika', 'Bahasa'],
    var_name='Mata Pelajaran', value_name='Nilai'
)

nilai_long
```

[79] ✓ 0.0s

	Nama	Mata Pelajaran	Nilai
0	Andi	Matematika	88
1	Budi	Matematika	75
2	Citra	Matematika	90
3	Andi	Bahasa	85
4	Budi	Bahasa	80
5	Citra	Bahasa	88

```
jumlah = lambda x, y: x + y
kali = lambda x: x * 2
print('lambda jumlah:', jumlah(4, 6))
print('lambda kali:', kali(5))

nilai['Kategori'] = nilai['Matematika'].apply(
    lambda skor: 'A' if skor >= 85 else ('B' if skor >= 75 else 'C')
)

nilai
```

[78]

lambda jumlah: 10
lambda kali: 10

	Nama	Matematika	Bahasa	Kategori
0	Andi	88	85	B
1	Budi	75	80	B
2	Citra	90	88	A

Gambar 17: Pivot table, melt, dan lambda function pada pandas.

Membaca file CSV dari Google Drive

Jika dataset ada di Google Drive, jalankan sel berikut lalu sesuaikan lokasi file.

```
# Hubungkan Google Drive jika notebook dijalankan di Google Colab
try:
    from google.colab import drive
    drive.mount('/content/drive')
    folder_data = '/content/drive/MyDrive'
    print('Mode Google Colab aktif')
except ModuleNotFoundError:
    from pathlib import Path
    folder_data = str(Path.cwd())
    print('Mode lokal VS Code aktif; gunakan file di folder notebook')
```

[30] ✓ 0.0s

Mode lokal VS Code aktif; gunakan file di folder notebook

```
# Ganti nama/path dataset sesuai lokasi file
from pathlib import Path
path_dataset = Path(folder_data) / 'dataset.csv'
if path_dataset.exists():
    dataset = pd.read_csv(path_dataset)
    display(dataset.head())
else:
    print(f'Dataset belum ditemukan: {path_dataset}')

# Contoh membaca CSV lokal yang dibuat dari DataFrame
contoh_path = Path(folder_data) / 'contoh_karyawan.csv'
data_karyawan.to_csv(contoh_path, index=False)
dataset_contoh = pd.read_csv(contoh_path)
dataset_contoh.head()
```

[31] ✓ 0.0s

Dataset belum ditemukan: [/home/hafidzprasetya/Kuliah/PPD/pertemuan-1/dataset.csv](#)

	Nama	Pekerjaan	Umur	Gaji
0	Andi	Programmer	23	8000000
1	Budi	Analisis	28	7000000
2	Citra	Programmer	25	9000000
3	Deni	Analisis	31	7500000
4	Eka	Desainer	24	6500000

Gambar 18: Pembacaan file CSV dan penampilan data menggunakan pandas.

3.2 Tugas dan Analisis Praktikum

3.2.1 Tugas 1: Array Acak dan Reshape

Saya membuat array NumPy berukuran 4×4 berisi bilangan acak dari 0 sampai 50, kemudian mengubah bentuknya menjadi 2×8 . Dari hasil eksekusi, elemen yang lebih besar dari 25 adalah 37, 43, dan 45. Proses ini menunjukkan bahwa operasi Boolean indexing dapat digunakan untuk memilih elemen berdasarkan kondisi.

Tugas 1

Buat array NumPy 2D berukuran 4x4 berisi angka acak 0-50, ubah menjadi array 2x8,

```
# Tulis jawabanmu di sini
# np.random.seed(1)
# tugas_1 = ...
```

[31]

> v

```
# Solusi referensi Tugas 1
np.random.seed(1)
tugas_1 = np.random.randint(0, 51, size=(4, 4))
tugas_1_2x8 = tugas_1.reshape(2, 8)
print('Array 4x4:\n', tugas_1)
print('Array 2x8:\n', tugas_1_2x8)
print('Elemen > 25:', tugas_1_2x8[tugas_1_2x8 > 25])
```

[32]

```
✓ 0.0s
...
Array 4x4:
[[37 43 12  8]
 [ 9 11  5 15]
 [ 0 16  1 12]
 [ 7 45  6 25]]
Array 2x8:
[[37 43 12  8  9 11  5 15]
 [ 0 16  1 12  7 45  6 25]]
Elemen > 25: [37 43 45]
```

Gambar 19: Hasil Tugas 1: pembuatan array acak 4×4, reshape menjadi 2×8, dan pemilihan elemen lebih dari 25.

3.2.2 Tugas 2: Groupby dan Aggregation Gaji

Saya membuat DataFrame karyawan kemudian mengelompokkan data berdasarkan pekerjaan. Hasil aggregation menunjukkan nilai maksimum, minimum, dan standar deviasi gaji pada setiap kelompok. Kelompok Programmer memiliki gaji maksimum Rp9.000.000 dan minimum Rp8.000.000, sedangkan kelompok Analis memiliki gaji maksimum Rp7.500.000 dan minimum Rp7.000.000. Kelompok Desainer hanya memiliki satu data sehingga standar deviasinya bernilai NaN.

Tugas 2

Buat DataFrame karyawan dengan kolom `Nama`, `Pekerjaan`, dan `Gaji`. Kelompokkan berdasarkan `Pekerjaan`, lalu hitung maksimum, minimum, dan standar deviasi `Gaji` menggunakan `agg()`.

```
# Tulis jawabanmu di sini
# tugas_2 = pd.DataFrame(...)
# hasil_tugas_2 = tugas_2.groupby('Pekerjaan').agg(...)
```

[11]

```
# Solusi referensi Tugas 2
tugas_2 = pd.DataFrame({
    'Nama': ['Ana', 'Bima', 'Caca', 'Dodi', 'Evi'],
    'Pekerjaan': ['Programmer', 'Analisis', 'Programmer', 'Analisis', 'Desainer'],
    'Gaji': [8000000, 7000000, 9000000, 7500000, 6500000]
})
hasil_tugas_2 = tugas_2.groupby('Pekerjaan').agg(
    Gaji_maksimum=('Gaji', 'max'),
    Gaji_minimum=('Gaji', 'min'),
    Gaji_std=('Gaji', 'std')
)
display(tugas_2)
display(hasil_tugas_2)
```

[11] ✓ 0.0s

	Nama	Pekerjaan	Gaji
0	Ana	Programmer	8000000
1	Bima	Analisis	7000000
2	Caca	Programmer	9000000
3	Dodi	Analisis	7500000
4	Evi	Desainer	6500000

	Gaji_maksimum	Gaji_minimum	Gaji_std
Pekerjaan			
Analisis	7500000	7000000	353553.390593
Desainer	6500000	6500000	NaN
Programmer	9000000	8000000	707106.781187

Gambar 20: Hasil Tugas 2: nilai maksimum, minimum, dan standar deviasi gaji berdasarkan pekerjaan.

3.2.3 Tugas 3: Merge DataFrame Karyawan dan Gaji

Saya menggabungkan DataFrame karyawan dan DataFrame gaji menggunakan `merge()` dengan parameter `left_on='Emp_id'`, `right_on='ID'`, dan `how='left'`. Hasilnya, data E01 dan E02 berhasil memperoleh informasi gaji, sedangkan E03 tetap ditampilkan dengan nilai NaN karena tidak memiliki pasangan ID pada DataFrame gaji.

Tugas 3

Buat DataFrame karyawan dengan kolom `Emp_id`, `Nama`, dan `Divisi`, serta DataFrame gaji dengan kolom `ID`, `Gaji`, dan `Status`. Masing-masing minimal tiga baris. Gabungkan dengan `merge()` menggunakan `left_on='Emp_id'`, `right_on='ID'`, `how='left'`, dan `suffixes`.

```
# Tulis jawabanmu di sini
# karyawan_tugas = pd.DataFrame(...)
# gaji_tugas = pd.DataFrame(...)
# hasil_tugas_3 = pd.merge(...)
```

```
# Solusi referensi Tugas 3
karyawan_tugas = pd.DataFrame({
    'Emp_id': ['E01', 'E02', 'E03'],
    'Nama': ['Ana', 'Bima', 'Caca'],
    'Divisi': ['Data', 'IT', 'Data']
})
gaji_tugas = pd.DataFrame({
    'ID': ['E01', 'E02', 'E04'],
    'Gaji': [8000000, 7000000, 6000000],
    'Status': ['Tetap', 'Kontrak', 'Tetap']
})
hasil_tugas_3 = pd.merge(
    karyawan_tugas, gaji_tugas,
    left_on='Emp_id', right_on='ID', how='left',
    suffixes=('_karyawan', '_gaji')
)
display(hasil_tugas_3)
```

	Emp_id	Nama	Divisi	ID	Gaji	Status
0	E01	Ana	Data	E01	8000000.0	Tetap
1	E02	Bima	IT	E02	7000000.0	Kontrak
2	E03	Caca	Data	NaN	NaN	NaN

Gambar 21: Hasil Tugas 3: penggabungan DataFrame karyawan dan gaji menggunakan `merge()` dengan left join.

Kesimpulan

Berdasarkan praktikum yang telah saya lakukan, saya memahami beberapa hal berikut:

1. NumPy digunakan untuk mengolah array berdimensi satu atau lebih secara efisien. Saya mempelajari indexing, slicing, reshape, penggabungan, pencarian, pengurutan, bilangan acak, serta operasi numerik pada array.
2. Fungsi `len()` pada NumPy hanya menunjukkan jumlah elemen pada sumbu pertama, sedangkan atribut `shape` menunjukkan ukuran array pada seluruh dimensinya. Sebagai contoh, array dengan bentuk (4, 2, 3) memiliki nilai `len()` sebesar 4 dan `shape` (4, 2, 3).
3. Fungsi `groupby()` pada pandas digunakan untuk mengelompokkan data berdasarkan kategori tertentu. Setelah dikelompokkan, data dapat diringkas menggunakan fungsi aggregation seperti `mean()`, `sum()`, `min()`, `max()`, dan `std()`.
4. Fungsi `merge()` menggabungkan DataFrame berdasarkan kolom kunci, sedangkan `join()` umumnya menggabungkan DataFrame berdasarkan indeks. Fungsi `concat()` dapat menggabungkan data secara vertikal maupun horizontal.

Dengan demikian, praktikum ini memberikan dasar yang penting bagi saya untuk melakukan pengolahan dan analisis data menggunakan Python.

Daftar Pustaka

- [1] NumPy Developers, “NumPy: The Absolute Basics for Beginners,” *NumPy User Guide*. [Online]. Tersedia: https://numpy.org/doc/stable/user/absolute_beginners.html.
- [2] The pandas Development Team, “10 Minutes to pandas,” *pandas User Guide*. [Online]. Tersedia: https://pandas.pydata.org/docs/user_guide/10min.html.
- [3] The pandas Development Team, “Group by: split-apply-combine,” *pandas User Guide*. [Online]. Tersedia: https://pandas.pydata.org/docs/user_guide/groupby.html.
- [4] The pandas Development Team, “Merge, join, concatenate and compare,” *pandas User Guide*. [Online]. Tersedia: https://pandas.pydata.org/docs/user_guide/merging.html.
- [5] The pandas Development Team, “Reshaping and pivot tables,” *pandas User Guide*. [Online]. Tersedia: https://pandas.pydata.org/docs/user_guide/reshaping.html.
- [6] The pandas Development Team, “IO tools,” *pandas User Guide*. [Online]. Tersedia: https://pandas.pydata.org/docs/user_guide/io.html.
- [7] A. Muhardian, “Memahami: Apa itu Numpy pada Python?,” *Petani Kode*, 12 November 2022. [Online]. Tersedia: <https://www.petanikode.com/python-numpy/>. [Diakses 24 Agustus 2026].